

# Operations Manual

---

## OTS Manager CLI & Batch

---

Management of identities, groups, and QR Codes via REST API on OpenTAKServer

---

### Document Control

| Field     | Information                                    |
|-----------|------------------------------------------------|
| Document  | Operations Manual — OTS Manager CLI & Batch    |
| Version   | <b>1.0</b>                                     |
| Author    | <b>Orlando Nascimento Santos</b>               |
| Email     | <b>onascimento@gmail.com</b>                   |
| Platform  | OpenTAKServer (OTS)                            |
| Interface | Python, CLI, and REST API                      |
| Language  | English (translated from Brazilian Portuguese) |

**Objective.** This manual presents a installation, configuration, operation, batch provisioning, QR Code generation, diagnostics, and best practices do utilitário `ots_manager.py`.

## Table of Contents

---

1. Overview
2. Architecture and operational flow
3. Prerequisites and installation
4. Configuration via environment variables
5. Security and credential protection
6. REST API mapping
7. CLI commands
8. Batch provisioning
9. QR Codes and access validity
10. Operational procedures
11. Error handling and diagnostics
12. Best practices and limitations
13. Operations checklist
14. Quick reference
15. Appendix: code structure

# 1. Overview

---

**OTS Manager** is a Python utility for automating OpenTAKServer administration. It encapsulates the operations required to:

- authenticate to the backend via `/api/login`;
- maintain an HTTP session with cookies;
- retrieve and send CSRF tokens required by Flask-WTF/Flask-Security;
- send the `Referer` and `Origin` headers;
- create groups;
- create users with mandatory password confirmation;
- associate users with groups in the `IN` and `OUT` directions;
- generate QR Code strings and PNG images for Android and iPhone;
- process multiple users from a JSON file;
- consolidate the processing results into a JSON report.

## 1.1 Operating model

The recommended workflow is:

1. configure `OTS_URL`, `OTS_USER` e `OTS_PASS`;
2. validate connectivity with the server;
3. perform the login;
4. create the required groups;
5. create the user;
6. associate the user with groups in `IN` e `OUT`;
7. generate the QR Code for the correct ecosystem;
8. store the PNG and record the result;
9. validate activation on the ATAK/iTAK device.

## 1.2 Ecosystems suportados

| Application    | Endpoint                         | Operation | Result                       |
|----------------|----------------------------------|-----------|------------------------------|
| Android / ATAK | <code>/api/atak_qr_string</code> | POST      | QR Code configuration string |
| iPhone / iTAK  | <code>/api/itak_qr_string</code> | GET       | iOS configuration string     |

# 2. Architecture and operational flow

---

The script uses `requests.Session()` to preserve cookies and headers between calls. After login, the CSRF token can be obtained from the cookies `csrf_token`, `csrf_access_token` ou `XSRF-TOKEN`. If the server returns the token in the response body, the utility also attempts to retrieve it from the JSON.

## 2.1 Authentication flow

```
ots_manager.py
|
| POST /api/login
v
OpenTAKServer
|
| cookies de sessão + token CSRF
v
Sessão requests.Session()
|
+--> Referer / Origin
+--> X-CSRFToken / X-CSRF-TOKEN
+--> operações protegidas
```

## 2.2 Applied headers

The expected global headers are:

```
Content-Type: application/json
Referer: <OTS_URL>/
Origin: <OTS_URL>
```

Para protected operations, o token é enviado adicionalmente como:

```
X-CSRFToken: <token>
X-CSRF-TOKEN: <token>
```

## 3. Prerequisites and installation

### 3.1 Requirements

- Python 3.9 or higher;
- network access to OpenTAKServer;
- credentials for an account authorized to create users and groups;
- write permissions in the working directory;
- `pip` available in the Python environment.

### 3.2 Dependency installation

```
python3 -m venv .venv
. .venv/bin/activate
python -m pip install --upgrade pip
pip install requests qrcode pillow
```

The libraries have the following responsibilities:

| Library  | Function                                 |
|----------|------------------------------------------|
| requests | HTTP communication, session, and cookies |
| qrcode   | QR Code image generation                 |
| Pillow   | Image backend used by qrcode             |

### 3.3 Suggested organization

```
ots-manager/
├── ots_manager.py
├── usuarios.json
├── qrcodes/
├── resultado_qr_codes.json
└── .venv/
```

Do not version files containing passwords, tokens, QR strings, or real personal data.

## 4. Configuration via environment variables

Set the parameters before execution:

```
export OTS_URL="http://164.152.57.64:5000"
export OTS_USER="admin"
export OTS_PASS="sua_senha_aqui"
```

The script removes the trailing slash from `OTS_URL`. If the variables are absent, the default values are:

| Variable | Code default          | Recommendation                 |
|----------|-----------------------|--------------------------------|
| OTS_URL  | http://localhost:5000 | Always specify explicitly      |
| OTS_USER | admin                 | Use a named or service account |
| OTS_PASS | admin_password        | Never rely on the default      |

### 4.1 Connectivity test

```
curl -i "$OTS_URL/"
```

If the service requires HTTPS, use an `https://` and validate the certificate. Avoid disabling TLS validation in production.

### 4.2 Running without exposing the password in shell history

Whenever possible, provide the password through a secure environment mechanism. At a minimum, avoid placing the password directly in shared commands or versioned scripts:

```
read -r -s OTS_PASS
export OTS_PASS
python ots_manager.py --help
```

## 5. Security and credential protection

**Warning:** the original example uses HTTP and an IP address. In production, prefer HTTPS with a valid certificate, a restricted network, and an account with the least privilege necessary.

Best practices:

- do not store `OTS_PASS` no repositório;
- do not share `resultado_qr_codes.json` without removing QR strings;
- protect generated PNG files because they may enable device provisioning;
- use service accounts with password rotation;
- restrict server access using a firewall or VPN;
- record operations without recording passwords or tokens;
- validate the destination before running destructive or provisioning commands;
- remove temporary files containing credentials after use.

## 6. REST API mapping

| Operation      | Endpoint                         | Method | Main data                                                                                                                             | CSRF/Referer                   |
|----------------|----------------------------------|--------|---------------------------------------------------------------------------------------------------------------------------------------|--------------------------------|
| Authentication | <code>/api/login</code>          | POST   | <code>username</code> , <code>password</code>                                                                                         | Sessão inicial                 |
| Create group   | <code>/api/groups</code>         | POST   | <code>name</code>                                                                                                                     | Sim                            |
| Create user    | <code>/api/user/add</code>       | POST   | <code>username</code> , <code>password</code> ,<br><code>confirm_password</code> , <code>email</code> ,<br><code>administrator</code> | According to OTS configuration |
| Link group     | <code>/api/users/groups</code>   | PUT    | <code>username</code> , <code>groups[]</code> , <code>direction</code>                                                                | According to OTS configuration |
| QR Android     | <code>/api/atak_qr_string</code> | POST   | <code>username</code> , <code>exp</code> , <code>nb</code> , <code>max</code>                                                         | Sim                            |
| QR iPhone      | <code>/api/itak_qr_string</code> | GET    | defined by the server                                                                                                                 | Authenticated session          |

### 6.1 Required and optional fields

The table below consolidates the fields accepted pelo `ots_manager.py`. Fields marked as **opcionais** may be omitted; in which case the script uses the CLI default or lets OpenTAKServer apply its own policy.

| Operation                               | Required fields                                             | Optional fields and default behavior                                                                                                                                                                       |
|-----------------------------------------|-------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Authentication — /<br>api/login         | username, password                                          | None. They are obtained from OTS_USER e OTS_PASS ; o código possui valores padrão, mas recomenda-se configure ambos explicitamente.                                                                        |
| Create group — /<br>api/groups          | name                                                        | Nenhum.                                                                                                                                                                                                    |
| Create user — /<br>api/user/add         | username, password,<br>confirm_password                     | email <b>is not required</b> and may be omitted or left empty; administrator is optional and defaults to false. O confirm_password é preenchido automaticamente pelo script com o mesmo valor de password. |
| Link group — /api/<br>users/groups      | username, groups[]<br>with at least one group,<br>direction | Na CLI, direction is optional and defaults to BOTH , performing the IN e OUT .                                                                                                                             |
| QR Android —<br>/api/<br>atak_qr_string | username                                                    | exp e max são <b>opcionais</b> . nbf is calculated automatically only when exp é informado. Se exp e max are omitted or null , are not sent and the server default policy applies.                         |
| QR iPhone — /api/<br>itak_qr_string     | Authenticated session                                       | Não há campos informados na requisição GET ; a configuração é retornada pelo servidor.                                                                                                                     |
| CLI create-user                         | --username, --<br>password                                  | --email, --groups, --admin, --app, --exp, --max e --save-qr are optional. --app assume android; --admin defaults to false.                                                                                 |
| CLI qr                                  | --username                                                  | --app, --exp, --max e --save-qr are optional. --app assume android.                                                                                                                                        |
| CLI create-group                        | --name                                                      | Nenhum.                                                                                                                                                                                                    |
| CLI link                                | --username, --group                                         | --direction is optional and defaults to BOTH .                                                                                                                                                             |
| CLI batch                               | --file                                                      | --output is optional and defaults to resultado_qr_codes.json . Dentro de cada registro, username e password são obrigatórios; email, administrator, groups, app, expiration e max_uses are optional.       |

## Summary of batch user fields

| Field JSON    | Required?  | Notes                                                                                                                                                      |
|---------------|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| username      | <b>Sim</b> | User identifier in OTS.                                                                                                                                    |
| password      | <b>Sim</b> | Initial password; the script also sends <code>confirm_password</code> com o mesmo valor.                                                                   |
| email         | <b>Não</b> | May be omitted, set to <code>null</code> or an empty string, depending on the OTS installation validation. The script sends it as empty when not provided. |
| administrator | <b>Não</b> | Assume <code>false</code> when omitted.                                                                                                                    |
| groups        | <b>Não</b> | Group list; when provided, each group is associated in <code>IN</code> e <code>OUT</code> .                                                                |
| app           | <b>Não</b> | Assume <code>android</code> ; also accepts <code>iphone</code> .                                                                                           |
| expiration    | <b>Não</b> | Days or date <code>YYYY-MM-DD</code> ; when absent or <code>null</code> , no expiration is sent.                                                           |
| max_uses      | <b>Não</b> | Limite de ativações; when absent or <code>null</code> , no limit is sent.                                                                                  |

**Important:** `email`, `expiration` e `max_uses` are not required fields. The absence of these fields does not prevent user creation or QR Code generation; in these cases, the server applies its default values and policies. However, `username` e `password` are essential for each user. The field `confirm_password` is required by the API, but is generated automatically by the script and does not need to be supplied separately in the CLI or JSON file.

## 6.2 Successful responses

Depending on the operation, the utility treats status codes `200`, `201` e `204`. An existing group or user may be treated as an idempotent condition when the response `400` contains a duplication indication, como `exists`.

## 6.2 Group directions

The association is performed separately:

- `IN`: messages received by the user;
- `OUT`: messages sent by the user;
- `BOTH`: CLI shortcut that executes two requests, one for each direction.

For bidirectional tactical traffic, always validate `IN` e `OUT`.

# 7. CLI commands

## 7.1 Help

```
python ots_manager.py --help
python ots_manager.py create-user --help
python ots_manager.py qr --help
```

## 7.2 Create user e gerar QR Code Android

```
python ots_manager.py create-user \  
-u "piloto1" \  
-p "Senha123!" \  
-g CSAR \  
--app android
```

The command creates the group, creates the user, associates the group in the **IN** e **OUT**, requests the string from OTS, and saves a PNG with a default name such as **piloto1\_android.png**.

## 7.3 Create user e gerar QR Code iPhone

```
python ots_manager.py create-user \  
-u "piloto2" \  
-p "Senha123!" \  
-g CSAR \  
--app iphone
```

## 7.4 Provide email and administrator profile

```
python ots_manager.py create-user \  
-u "operador" \  
-p "SenhaForte!" \  
-e "operador@empresa.com" \  
--admin \  
--app android
```

Use **--admin** somente quando a função operacional realmente exigir privilégios administrativos.

## 7.5 Validity and activation limit

```
python ots_manager.py create-user \  
-u "convidado" \  
-p "Senha123!" \  
-g Visitantes \  
--app android \  
--exp 30 \  
--max 1
```

**--exp 30** represents 30 days from the time of generation. A date in the format can also be used: **YYYY-MM-DD**:

```
python ots_manager.py qr -u "piloto1" --app android --exp 2026-12-31 --max 2
```

When **--exp** e **--max** are omitted, these fields are not sent and the server applies its default policies.



## 7.6 Generate a QR Code for an existing user

```
python ots_manager.py qr -u "piloto1" --app android
python ots_manager.py qr -u "piloto2" --app iphone --save-qr piloto2_ios.png
```

## 7.7 Create group isolado

```
python ots_manager.py create-group -n "Patrulha"
```

## 7.8 Link a user to a group

```
python ots_manager.py link -u "piloto1" -g "Patrulha"
```

Specific direction:

```
python ots_manager.py link -u "piloto1" -g "Patrulha" --direction IN
python ots_manager.py link -u "piloto1" -g "Patrulha" --direction OUT
```

# 8. Batch provisioning

---

## 8.1 File `usuarios.json`

```
[
  {
    "username": "operador_alpha",
    "password": "SenhaForte123!",
    "email": "alpha@empresa.com",
    "administrator": false,
    "groups": ["CSAR", "Resgate"],
    "app": "android"
  },
  {
    "username": "operador_bravo",
    "password": "SenhaForte456!",
    "email": "bravo@empresa.com",
    "administrator": false,
    "groups": ["CSAR"],
    "app": "iphone"
  },
  {
    "username": "operador_temporario",
    "password": "TempPassword789!",
    "email": "temp@empresa.com",
    "administrator": false,
    "groups": ["Operacoes"],
    "app": "android",
    "expiration": 30,
    "max_uses": 1
  }
]
```

As chaves `expiration` e `max_uses` are optional. Se estiverem ausentes ou com valor `null`, as restrições não serão enviadas ao servidor.

## 8.2 Execution

```
python ots_manager.py batch \  
-f usuarios.json \  
-o resultado_qr_codes.json
```

Processing:

1. creates all groups found;
2. processes each user;
3. associates groups in `IN` e `OUT`;
4. converts validity to Unix Epoch when necessary;
5. requests the QR Code;
6. saves PNGs in `qr_codes/`;
7. exports the consolidated report.

## 8.3 Report structure

```
[  
  {  
    "username": "operador_alpha",  
    "app": "android",  
    "max_uses": "padrão do servidor",  
    "expiration": "padrão do servidor",  
    "qr_string": "string retornada pelo OTS",  
    "qr_image": "qr_codes/operador_alpha_android.png"  
  }  
]
```

The report must be treated as sensitive material. In real environments, consider generating an operational version without the `qr_string` key for sharing.

# 9. QR Codes and access validity

## 9.1 Android

The Android endpoint receives `username`. The optional fields are:

| Field            | Meaning                            |
|------------------|------------------------------------|
| <code>exp</code> | expiration instant in Unix Epoch   |
| <code>nbf</code> | instant from which the QR is valid |
| <code>max</code> | activation limit                   |

When existe expiração, o script calcula `nbf` as the current instant in UTC.

## 9.2 iPhone

The iPhone endpoint is queried using `GET` and directly returns the iTAK configuration string. The response may be JSON, with keys such as `qr_string` ou `itak_qr_string`, or plain text.

## 9.3 Date conversion

A função `parse_expiration` aceita:

- number of days, such as `30`;
- absolute date, such as `2026-12-31`;
- empty values or values equivalent to `none`, `null`, `eterno` e `infinito`, treated as no restriction.

Absolute dates are interpreted in UTC. Confirm the timezone and server policy before using expiration dates in critical operations.

# 10. Operational procedures

---

## 10.1 Provision a new team

1. confirm the OTS address and operating account;
2. list the required groups;
3. review the JSON with another operator;
4. run the batch during an authorized window;
5. check the report and PNGs;
6. distribute each QR Code only to the correct recipient;
7. test login and CoT traffic on the device;
8. record the execution and validity date.

## 10.2 Reissue a QR Code

```
python ots_manager.py qr \  
--username "usuario_existente" \  
--app android \  
--save-qr "reemitido_usuario_android.png"
```

Confirm whether reissuing invalidates the previous QR Code according to OTS policy.

## 10.3 Validate post-provisioning connectivity

On the device:

- import the QR Code into the corresponding application;
- confirm the server address;
- confirm that the user can authenticate;
- test receipt of a CoT message;
- test sending a CoT message;
- validate that the expected groups are applied.

## 11. Error handling and diagnostics

---

### Connection failure

**Symptom:** erro de conexão ou timeout.

**Checks:**

```
curl -i "$OTS_URL/"
getent hosts <hostname>
```

Confirm the IP, port, firewall, VPN, and that the OTS service is active.

### Authentication failure

**Symptom:** mensagem `Falha na autenticação` com status HTTP diferente de `200` ou `201`.

Check the username, password, URL, login method, and whether the account is enabled.

### HTTP 400 ao criar grupo ou usuário

This may indicate invalid data or duplication. O script reconhece duplicidade quando o corpo contém `exists`; otherwise, read the complete response and correct the submitted fields.

### HTTP 401 ou 403 in a protected operation

Check:

- whether login actually created the session;
- whether the CSRF token exists in the cookies or response;
- whether the headers `X-CSRFToken` e `X-CSRF-TOKEN` were sent;
- se `Referer` e `Origin` match the `OTS_URL`;
- whether the account has permission for the operation;
- whether the session has expired.

### Empty or invalid QR Code

Confirm the selected endpoint ( `android` ou `iphone` ), the user, HTTP status, and response format. Preserve the returned body during diagnostics without exposing tokens in public logs.

### PNG not generated

Confirm the installation of `qrcode` e `pillow`, the existence of the destination directory and write permissions:

```
python -c "import qrcode, PIL; print('dependências OK')"
```

## Partially completed batch

The batch may create some resources before failing on a later record. Do not rerun blindly. Compare the report, check which users and groups exist, and handle duplicates in a controlled manner.

## 12. Best practices and limitations

---

- Create a secure backup do arquivo de entrada antes de uma execução em lote.
- Use stable usernames and documented conventions.
- Avoid spaces and ambiguous characters in identifiers.
- Test first with a laboratory account.
- Do not distribute QR Codes through public channels.
- Record the generation date, validity, and person responsible for delivery.
- Always validate both group directions when there is bidirectional traffic.
- O modo `BOTH` executa duas requisições; uma falha pode deixar a associação incompleta.
- Handling “already exists” does not replace checking the current state on the server.
- The endpoint and response format may vary depending on the OpenTAKServer version.
- The original script does not implement transactional rollback. For critical batches, use smaller stages and subsequent reconciliation.
- The password appears on the command line in the examples; in production, prefer a secure input mechanism.

### 12.1 Notes de manutenção do código

When transcribing or updating the source code, validate the indentation of the block `get_qr_string`, especially the branches `elif app_type == "iphone"` e `else`. Run a syntax compilation before use:

```
python -m py_compile ots_manager.py
```

It is also recommended to add tests for `parse_expiration`, seleção dos endpoints, tratamento de respostas JSON/texto e montagem dos payloads.

## 13. Operations checklist

---

### Before execution

- [ ] `OTS_URL` aponta para o ambiente correto.
- [ ] O acesso de rede foi validado.
- [ ] A conta tem permissão suficiente.
- [ ] A senha não está versionada.
- [ ] O JSON foi validado e revisado.
- [ ] Os nomes de grupos e usuários estão corretos.
- [ ] `app` é `android` ou `iphone`.
- [ ] Expiração e limite de uso foram conferidos.

## After execution

- [ ] Login was successful.
- [ ] Groups were created or confirmed.
- [ ] Users were created or confirmed.
- [ ] Each group was associated in **IN** e **OUT**.
- [ ] The QR string was returned.
- [ ] The PNG was generated and opened for review.
- [ ] The JSON report was saved in a protected location.
- [ ] The device was tested in both communication directions.

## 14. Quick reference

---

```
# Instalação
pip install requests qrcode pillow

# Configuração
export OTS_URL="http://servidor-ots:5000"
export OTS_USER="admin"
export OTS_PASS="..."

# Usuário Android
python ots_manager.py create-user -u piloto1 -p 'Senha123!' -g CSAR --app android

# Usuário iPhone
python ots_manager.py create-user -u piloto2 -p 'Senha123!' -g CSAR --app iphone

# QR existente
python ots_manager.py qr -u piloto1 --app android --save-qr piloto1.png

# Grupo
python ots_manager.py create-group -n Patrulha

# Associação bidirecional
python ots_manager.py link -u piloto1 -g Patrulha

# Lote
python ots_manager.py batch -f usuarios.json -o resultado_qr_codes.json

# Validação sintática
python -m py_compile ots_manager.py
```

## 15. Appendix: code structure

---

O arquivo `ots_manager.py` está organizado pelas funções abaixo:

| Function                          | Responsibility                                                               |
|-----------------------------------|------------------------------------------------------------------------------|
| <code>get_csrf_token()</code>     | Retrieves the CSRF token from the session                                    |
| <code>login()</code>              | Authenticates and updates headers                                            |
| <code>create_group()</code>       | Creates or confirms a group                                                  |
| <code>create_user()</code>        | Creates or confirms a user                                                   |
| <code>add_user_to_group()</code>  | Associates groups in <code>IN</code> , <code>OUT</code> ou <code>BOTH</code> |
| <code>parse_expiration()</code>   | Converts days/date to Unix Epoch                                             |
| <code>get_qr_string()</code>      | Gets the Android or iPhone configuration                                     |
| <code>save_qr_code_image()</code> | Generates and saves the PNG image                                            |
| <code>process_batch_list()</code> | Orchestrates batch provisioning                                              |
| <code>main()</code>               | Defines the CLI and dispatches commands                                      |

## Conclusion

O OTS Manager reduces manual tasks and standardizes OpenTAKServer provisioning. Automation must be used together with access control, credential protection, input-file review, and functional validation on the ATAK/iTAK device.

---

### OTS Manager Operations Manual — Version 1.0

Created by Orlando Nascimento Santos — [onascimento@gmail.com](mailto:onascimento@gmail.com)

Document prepared based on the specification and source code provided in the original manual.